개인 회고

진행하면서 막혔던 부분과 해결

막힌 부분	원인	해결
내 노트북에서 할지 코랩에서 할지 고민	데이터셋만 약 1.9GB(압축을 풀면 더 늘어남)이고, PaDiM은 학습 특징 `[630336, 1536]`을 GPU에 모으는 것만 약 3.9GB, 합칠 때는 두 배 가까이 필요하다. 내 노트북(GPU 8GB)으로 했으면 용량도 많이 차지하고 시간도 한참 걸렸을 것 같았다	코랩 Pro(T4 GPU)에서 했다. 원본이 리눅스 기준 코드라 코랩에서는 거의 그대로 돌아갔다
첫 셀부터 `import wget`에서 막힐 뻔함	원본은 `# !pip install wget`까지 주석 처리되어 있는데, 코랩에는 `wget` 패키지가 없다(실행해 보니 새로 설치됐다)	이 줄만 주석을 풀고, 바로 위에 원본 줄과 바꾼 이유를 주석으로 남겼다. 원본 코드 중 바꾼 곳은 이 한 줄뿐이다

실습을 통해 알게 된 것

이상 탐지 이론을 배우는 데서 멈추지 않고 직접 구현해 보면서, 학습에는 정상 데이터만 쓰는데도 불량 데이터가 왜 필요한지 알게 됐다. 불량 데이터는 학습이 아니라, 다 만든 모델이 불량을 잘 가려내는지 시험하는 테스트 단계에서 필요하다.

또 이상 탐지는 목적에 따라 재현율(불량을 놓치지 않는 것)과 정밀도(불량이라고 한 것이 진짜 불량인 것) 중 무엇을 높일지 골라야 한다는 것을 알았다. 실습 코드의 평가 도구는 F1이 가장 높은 임계값(PaDiM은 22.26)을 찾아 둘의 균형을 맞췄다. 반면 개인 실험에서 웹 판정 기준인 임계값 15로 테스트셋을 판정해 보니, 실제 불량 100장은 모두 잡아 재현율은 100%였지만 실제 정상 201장 중 135장도 불량으로 판정해서 정밀도가 크게 떨어졌다(불량 판정 235장 중 진짜 불량은 100장).

세 모델은 정상만 보고 "평소와 다른 정도"를 점수로 매긴다는 점은 같고, 점수를 매기는 방식이 다르다. PaDiM은 사진을 바둑판처럼 칸으로 나눠서 칸마다 "여기는 보통 이렇게 생겼다"를 외워 두고, 가장 이상한 칸의 점수를 사진의 점수로 쓴다. One-Class SVM은 사진 전체를 숫자 묶음 하나로 요약한 뒤 정상들을 감싸는 울타리를 치고, 울타리 밖이면 이상으로 본다. AutoEncoder는 사진을 작게 접었다가 다시 펴는 연습을 정상 사진으로만 시켜서, 잘 못 펴지는 사진을 이상으로 본다. 이번 실습에서는 위치까지 보는 PaDiM이 가장 잘했다(AUROC 0.962).

교수님이 수업에서 "loss만 보고 판단한 실험은 실패"라고 하신 것도 이번 실습으로 직접 확인했다. AutoEncoder의 러닝 커브를 그려 보니 학습 loss는 2.28에서 1.65까지 계속 줄었지만, 테스트 AUROC는 1에폭부터 30에폭까지 0.26 근처에서 전혀 오르지 않았다. loss만 봤다면 학습이 잘 되고 있다고 착각했을 것이다. 마지막 Sigmoid 하나만 빼고 같은 조건으로 다시 학습하니(before/after) AUROC가 0.92까지 올라서, 문제는 학습 횟수가 아니라 구조였다는 것을 알 수 있었다. 이어서 이 구조를 60에폭까지 학습하면서 검증용 데이터의 AUROC로 최적 에폭을 고르고, 그 모델을 최종 테스트용 데이터로 한 번만 평가했다. 시드 5개로 반복한 결과 최종 테스트 AUROC는 0.935 ± 0.006이었다. 테스트에 쓸 데이터로 모델을

고르면 숫자가 부풀려지기 때문에, 고르는 데이터와 평가하는 데이터를 나누는 것이 중요하다는 것도 알게 됐다.

개인 실험 요약

오늘 수업 시간에 주신 피드백을 바탕으로 다른 방법으로 더 시도해 봤습니다. 원본 코드는 건드리지 않았고, 실험 셀에는 모두 `[개인 실험]` 표시를 했습니다.

1. **불량 데이터 비율**: 원본 코드는 정상 사진 수만 세서 불량은 몇 장인지 궁금했습니다. 그래서 카테고리마다 불량 폴더도 세어 보니 12개 카테고리 모두 불량이 100장, 불량 비율은 9~17%(전체 11.1%)였습니다. 이론에서 배운 대로, 현장(불량 1%)만큼은 아니어도 데이터 불균형이 있다는 걸 확인했습니다.
2. **PaDiM이 틀린 사진**: PaDiM은 어떤 사진을 틀리는지 궁금해서 점수가 가장 높은 실제 정상 3장과 가장 낮은 실제 불량 3장을 그려 봤습니다. 실제 정상 201장 중 19장을 불량으로, 실제 불량 100장 중 8장을 정상으로 판정했고, 불량으로 판정한 실제 정상 중에는 기판이 기울어진 사진이 있었습니다. 임계값 하나로 둘을 완벽히 나눌 수 없고, 촬영 조건도 중요하다고 느꼈습니다.
3. **웹 판정 기준 15**: 원본 Gradio 코드의 판정 기준 15가 맞는 값인지 궁금해서 테스트셋 전체를 15로 판정해 봤습니다. 실제 불량은 100% 잡았지만 실제 정상 201장 중 67.2%도 불량으로 판정했습니다. 기준이 너무 낮아 실제 정상까지 불량으로 판정하는 걸 확인해서, 수정이 필요하다고 생각했습니다.
4. **웹 화면에서의 판정**: 실제 웹 화면에서는 어떻게 판정하는지 보려고 Gradio에 사진 4장(평범한 정상, 기울어진 정상, 경계선 불량, 확실한 불량)을 올려 봤습니다. 4장 모두 비정상으로 판정했고(18.10 / 31.56 / 22.03 / 45.29점), 이물질 사진은 히트맵이 정확히 그 위치를 가리켰습니다. 3번에서 확인한 것처럼 웹 화면에서도 기준 15로는 실제 정상까지 불량으로 판정해, 제 역할을 못한다는 것을 확인했습니다.
5. **loss만 보면 안 되는 이유**: 교수님 조언대로 loss만 보지 않고 AUROC도 같이 보려고, AutoEncoder를 다시 학습하며 에폭마다 loss와 AUROC를 함께 기록했습니다(러닝 커브). 학습 loss는 2.28에서 1.65로 줄어 학습이 잘 되는 것처럼 보였지만, AUROC는 30에폭 내내 0.26 근처로 무작위(0.5)보다도 낮았습니다. 실제로는 불량을 가려내지 못하고 있었다는 걸 알았습니다.
6. **AutoEncoder가 안 된 원인**: AI에게 물어보니 마지막 Sigmoid를 빼 보라고 해서, Sigmoid만 빼고 같은 조건으로 다시 학습했습니다(before/after). AUROC가 0.26에서 0.92로 크게 올라, 학습 부족이 아니라 모델 구조가 문제였다는 걸 알게 됐습니다. 이유를 물어보니, 입력은 정규화로 약 -2.1~2.6 범위인데 Sigmoid는 출력을 0~1로 묶어서 처음부터 제대로 복원할 수 없는 구조였기 때문이었습니다.
7. **PaDiM 수치의 신뢰도**: 평소 AI 모델링을 할 때 시드를 여러 개로 돌려 평균을 내고 결과가 얼마나 흔들리는지 확인한다는 걸 알고 있어서, PaDiM을 시드 5개로 다시 학습해 봤습니다. 평균 0.960 ± 0.004로 시드가 바뀌어도 거의 흔들리지 않아, 0.962가 우연히 나온 수치가 아니라는 걸 확인했습니다.

8. **AutoEncoder 최적화**: AutoEncoder를 더 최적화하고 싶어서, 테스트셋을 검증용과 최종 테스트용으로 나누고 60에폭 동안 검증 AUROC로 최적 에폭을 고른 뒤 최종 테스트는 한 번만 평가했습니다(시드 5개). 이상 점수를 "오차가 큰 상위 1% 픽셀 평균"으로도 바꿔 봤습니다. 결과는 0.935 ± 0.006(고른 에폭 26~53)이었고, 상위 1% 점수는 0.933 ± 0.013으로 나아지지 않았습니다. AI에게 이유를 물어보니 AUROC가 30에폭 무렵 이미 0.92~0.94에서 멈춰서 에폭을 늘려도 성능이 거의 오르지 않는다는 것이었고, 9번에서 직접 확인했습니다. 또 고르는 데이터와 평가하는 데이터를 나눠야 숫자가 부풀려지지 않는다는 것도 알게 됐습니다.
9. **과적합 시점**: 과적합을 찾아보고 싶어 200에폭까지 학습해 봤습니다(시드 2개). 검증 loss가 가장 낮은 에폭이 199·193에폭이라 과적합은 아직 일어나지 않았습니다. 하지만 AUROC는 30에폭 이후 0.92~0.94에서 멈춰, 이 모델은 30~60에폭이면 충분하다고 생각했습니다.
10. **같은 조건에서 모델 비교**: 최적화한 AutoEncoder를 PaDiM과 같은 최종 테스트 151장으로 비교해 봤습니다(시드 5개). PaDiM 0.977 ± 0.003, AutoEncoder 0.935 ± 0.006으로 PaDiM 성능이 더 좋아, 이 작업에서는 PaDiM이 확실히 낫다고 느꼈습니다.
11. **임계값을 데이터로 정하기**: 3번에서 기준 15가 너무 낮았기 때문에, 검증 데이터로 임계값을 정하고 최종 테스트로 한 번만 확인했습니다. 검증에서 F1이 가장 높은 22.13으로 정하니 최종 테스트에서 재현율 98.0%, 정밀도 84.5%였습니다. 기준 15일 때(재현율 100%, 정밀도 40.3%)와 비교하면 실제 정상을 불량으로 판정한 사진이 101장 중 74장에서 9장으로 크게 줄었고, 실제 불량을 놓친 사진은 50장 중 1장이었습니다. 임계값을 낮추니 역시 정밀도는 떨어지고 재현율은 올라가는 것을 확인했고, 임계값은 데이터로 정해야겠다고 느꼈습니다.

어려웠던 것

AutoEncoder의 AUROC가 0.26이 나와서 처음에는 그냥 성능이 나쁜 줄 알았다. 그런데 공부해 보니 AUROC가 0.5보다 낮다는 것은 아무것도 모르는 게 아니라 정상과 불량을 거꾸로 맞혔다는 뜻이었다. 그래서 점수를 뒤집으면 1 - 0.264 = 약 0.74가 된다. 다만 뒤집어도 PaDiM(0.962)이나 One-Class SVM(0.824)보다 낮고, 원래 원인은 출력 범위(0~1)가 입력 범위(약 -2.1~2.6)와 달라 제대로 복원할 수 없는 구조라서, 뒤집는다고 좋은 모델이 되는 것은 아니다. 실제로 원인이었던 Sigmoid를 빼고 다시 학습하니 AUROC가 0.92로 올랐다.

좋았던 점

코랩으로 진행해서 학습이 빨리 돼 좋았다.

현장에 쓴다면

실제 공장에서 쓴다면 여러 임계값으로 테스트해 보고, 원하는 정밀도와 재현율을 확보하는 값을 고를 것 같다. 임계값 15처럼 정해 두면 실제 정상의 67%를 불량으로 판정하게 된다. 또 정상 제품이 불량으로 잘못 판정된 사진을 살펴보니 기판이 기울어져 촬영된 사진이 있었고, 위치마다 정상을 외우는 PaDiM에는 그것만으로도 이상해 보였을 가능성이 크다. 그래서 데이터를 모을 때부터 제품을 같은 위치·각도로 놓고 일관된 형태로 촬영할 것 같다. 실제로 개인 실험에서 임계값을 검증 데이터로 22.13으로 정하니, 최종 테

스트에서 실제 정상을 불량으로 판정하는 경우가 74장 → 9장(101장 중)으로 줄고 놓친 불량은 50장 중 1장이었다. 현장에서도 임계값은 감으로 정하지 않고 이렇게 데이터로 정할 것이다.

앞으로 해 볼 것

PaDiM은 무작위 요소 때문에 두 번 돌렸을 때 AUROC가 0.961, 0.962로 달라서, 시드 5개로 다시 돌려 평균을 내 보니 0.960 ± 0.004였다. 수치를 믿을 수 있다는 것은 확인했다. AutoEncoder도 시드 5개로 0.935 ± 0.006을 얻었다. 과적합 시점을 확인하려고 200에폭까지 늘려 보니, 검증 loss는 거의 끝까지 줄었지만(최저 199·193에폭) 검증 AUROC는 30에폭 이후 0.92~0.94에서 더 오르지 않았다. loss가 아니라 지표가 멈추는 지점(30~60에폭)에서 학습을 멈춰야 한다는 것을 숫자로 확인했다. 같은 최종 테스트 데이터(151장)로 비교해 보니 PaDiM 0.977 ± 0.003, AutoEncoder 0.935 ± 0.006으로 PaDiM이 더 좋았다. 앞으로는 검증 데이터를 더 늘려 최적 에폭과 임계값을 더 안정적으로 골라 보고 싶다.